

JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING & MANAGEMENT, NAGPUR



Education to Eternity

Department of Computer Science & Engineering

SESSION: 2025-2026 5th SEM- CSE

Subject: Blockchain Technology

UNIT-IV Notes

Presented By Khushboo Bisen

Asst. Prof. CSE/AI Dept.

Ethereum

Introduction

- Ethereum is a decentralized blockchain platform enabling smart contracts and DApps (Decentralized Applications).
- Proposed by Vitalik Buterin (2013) and launched in 2015.
- Unlike Bitcoin, which only supports transactions, Ethereum is programmable using Solidity language.

Ethereum Architecture

Main Layers:

1. Application Layer

- Interface for users and developers.
- Hosts DApps, wallets, and smart contracts.

2. Execution Layer

- Executes smart contracts using **Ethereum Virtual Machine (EVM)**.
- Each node runs EVM to ensure all get the same output.

3. Consensus Layer

- Maintains agreement on network state.
- Shifted from **Proof of Work (PoW)** to **Proof of Stake (PoS)** in 2022 (The Merge).

Ethereum Key Components

Component Description

Ether (ETH) Native cryptocurrency used to pay for transactions (“gas”).

Smart Contracts Self-executing programs deployed on the blockchain.

EVM (Ethereum Virtual Machine) Runtime environment executing smart contract bytecode.

Accounts Two types: Externally Owned (controlled by users) and Contract Accounts (controlled by code).

Ethereum Key Components

Component	Description
Ether (ETH)	Native cryptocurrency used to pay for transactions (“gas”).
Smart Contracts	Self-executing programs deployed on the blockchain.
EVM (Ethereum Virtual Machine)	Runtime environment executing smart contract bytecode.
Accounts	Two types: Externally Owned (controlled by users) and Contract Accounts (controlled by code).

Limitations

- High gas costs in congestion.
- Limited scalability (≈15 TPS).
- Smart contract bugs are permanent.
- Complex for non-technical users.

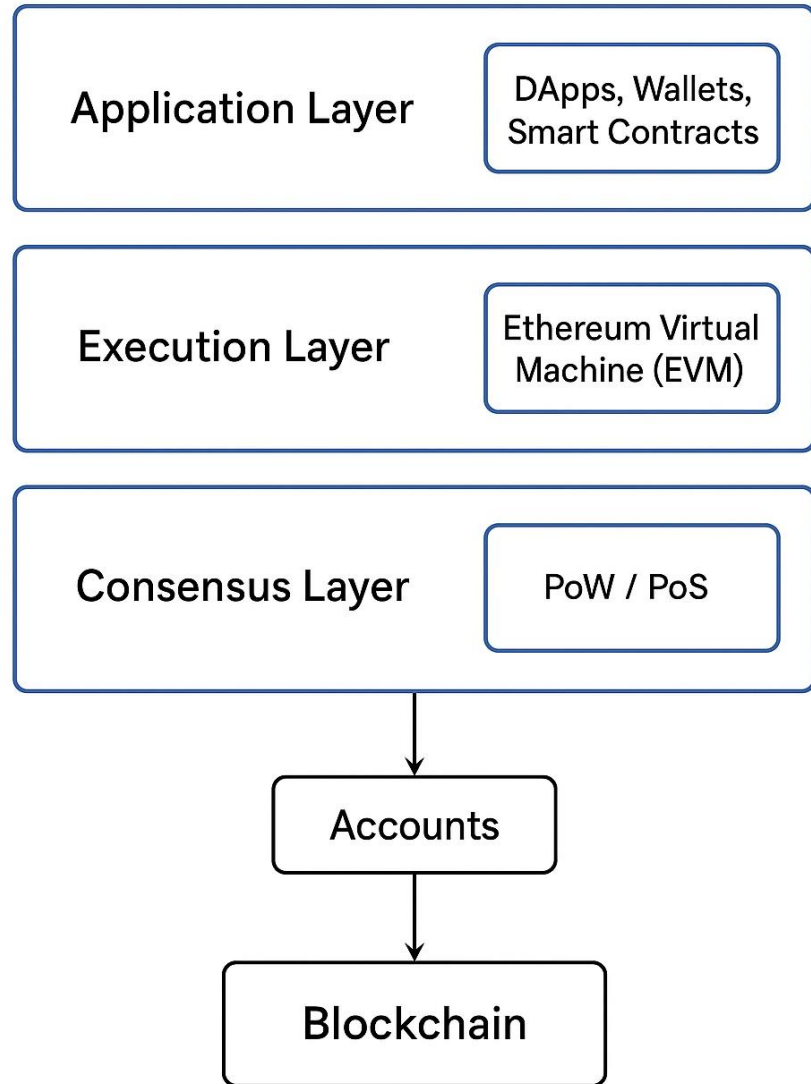
Comparison: Ethereum vs Bitcoin

Feature	Bitcoin	Ethereum
Purpose	Digital currency	Programmable platform
Launch	2009	2015
Language	Script	Solidity
Block Time	10 min	13 sec
Consensus	PoW	PoS
Flexibility	Limited	Turing complete
Use Cases	Payment	DApps, NFTs, DeFi

Real-World Applications

- **DeFi Platforms:** Aave, MakerDAO, Compound.
- **NFT Marketplaces:** OpenSea, Rarible.
- **DAOs:** Decentralized Governance.
- **Supply Chain & Identity:** Provenance, uPort.

Ethereum Architecture



Ethereum Virtual Machine (EVM)

Introduction

- The Ethereum Virtual Machine (EVM) is the runtime environment for executing smart contracts on the Ethereum blockchain.
- It acts as a decentralized computer that runs on thousands of nodes worldwide.
- Every Ethereum node runs the EVM to ensure all computations produce identical results, maintaining consensus.

Purpose of the EVM

- To execute smart contract code safely and deterministically.
- To ensure that execution produces the same state transition across all nodes.
- To provide a sandboxed environment, isolating contract code from the underlying system.

Working of the EVM

1. Smart Contract Deployment

- A developer writes a contract in **Solidity** or **Vyper**.
- The code is **compiled into EVM bytecode** and deployed on the blockchain.

2. Transaction Execution

- When a transaction invokes a contract, the bytecode runs inside the EVM.
- The EVM reads inputs, executes opcodes, and changes the blockchain state.

3. State Changes

- Execution may update **account balances, contract storage, or logs**.
- These updates are reflected in Ethereum's **global state**.

Structure of the EVM

Component Function

Stack Operates as a Last-In-First-Out (LIFO) memory for execution.

Memory Temporary data storage during execution (cleared after transaction).

Storage Permanent key-value storage associated with each smart contract.

Program Counter (PC) Keeps track of which instruction is being executed.

Gas Mechanism Measures computation cost; halts execution if gas runs out.

Advantages

- Deterministic and secure execution.
- Universal across all Ethereum nodes.
- Supports complex logic via smart contracts.
- Highly portable — runs on multiple blockchain networks.

Limitations

- Execution is slower than native machine code (interpreted bytecode).
- High gas costs for complex contracts.
- Limited access to external data (requires oracles).

Aspect	Description
--------	-------------

Full Form	Ethereum Virtual Machine
-----------	--------------------------

Role	Executes smart contracts deterministically
------	--

Language	Bytecode (compiled from Solidity/Vyper)
----------	---

EnvironmentIsolated (sandboxed)

Core Components Stack, Memory, Storage, Gas, Program Counter

Output New blockchain state after execution

Introduction to Solidity

Solidity is a brand-new programming language created by **Ethereum** which is the second-largest market of cryptocurrency by capitalization, released in the year 2015 and led by Christian Reitwiessner. Some key features of solidity are listed below:

- Solidity is a high-level programming language designed for implementing smart contracts.
- It is a statically typed object-oriented(contract-oriented) language.
- Solidity is highly influenced by Python, c++, and JavaScript which run on the Ethereum Virtual Machine(EVM).
- Solidity supports complex user-defined programming, libraries, and inheritance.
- Solidity is the primary language for blockchains running platforms.
- Solidity can be used to create contracts like voting, blind auctions, crowdfunding, multi-signature wallets, etc.

Operators and Control Statements in Solidity

Solidity, like other programming languages, uses operators to perform computations and control statements to manage the flow of execution.

Operators in Solidity

No.	Operator Type	Operators	Description	Example
1	Arithmetic Operators	+, -, *, /, %	Used for basic mathematical calculations.	uint c = a + b;
2	Relational (Comparison) Operators	==, !=, >, <, >=, <=	Compare two values and return true or false.	if (a > b)
3	Logical Operators	&&, ^, !		
4	Assignment Operators	=, +=, -=, *=, /=, %=	Used to assign or update variable values.	x += 3; // same as x = x + 3

5	Bitwise Operators	&, ` , ^, ~, <<, >>`	Perform bit-level operations on integer data.	
6	Conditional (Ternary) Operator	condition ? value1 : value2	Short-hand of if-else.	uint max = (a > b) ? a : b;
7	Increment / Decrement Operators	++, -	Increase or decrease a variable's value by 1.	i++;
8	Compound Operators	x += y, x -= y	Combine arithmetic and assignment in one.	balance -= amount;
9	New Operator	new	Used to create dynamic instances of contracts.	MyContract obj = new MyContract();
10	Delete Operator	delete varName	Resets a variable to its default value.	delete a; // sets a = 0

Control Statements in Solidity

Control statements decide **how and when** certain parts of code should execute.

No.		Control Statement	Syntax / Example	Description
1	if Statement	if(x > 10) { y = 5; }	Executes code only if condition is true.	
2	if-else Statement	if(a > b) max = a; else max = b;	Executes one of two blocks depending on condition.	
3	else-if Ladder	if(a == b) {...} else if(a > b) {...} else {...}	Checks multiple conditions in sequence.	
4	for Loop	for(uint i=0; i<5; i++) { sum += i; }	Executes code repeatedly for a set number of times.	
5	while Loop	while(i < 10) { i++; }	Executes code while condition remains true.	
6	do-while Loop	do { i++; } while(i < 5);	Executes once before checking the condition.	
7	break Statement	if(i == 5) break;	Terminates loop prematurely.	

8	continue Statement	if(i == 5) continue;	Skips current iteration and continues the loop.
9	return Statement	return value;	Returns value from a function.
10	assert / require Statements	require(x > 0); assert(balance >= amount);	Used to validate conditions; revert on failure.

Example Solidity Program

```
pragma solidity ^0.8.0;
```

```
contract Example {
```

```
    function operatorDemo(uint a, uint b) public pure returns (uint) {
```

```
        uint sum = a + b;        // Arithmetic
```

```
        uint max = (a > b) ? a : b; // Ternary
```

```
        return sum + max;
```

```
}
```

```
function controlDemo(uint x) public pure returns (string memory) {  
    if (x == 0) return "Zero";  
    else if (x % 2 == 0) return "Even";  
    else return "Odd";  
}
```

```
function loopDemo() public pure returns (uint) {  
    uint sum = 0;  
    for(uint i = 1; i <= 5; i++) {  
        if(i == 3) continue; // skip 3  
        sum += i;  
    }  
}
```

```
    return sum; // returns 12 (1+2+4+5)
}
}
```

Category

Examples

Operators (10)

Arithmetic, Relational, Logical, Assignment, Bitwise, Conditional,
Increment/Decrement, Compound, New, Delete

Control Statements (10)

if, if-else, else-if, for, while, do-while, break, continue, return,
assert/require